

Structural verification of index invariants in a running database

Andrey Borodin

Аннотация

A defect in an index access method does not announce itself. The index does not fail; it begins to answer incompletely, and a query whose answer exists in the table does not find it. The application keeps working, and the discrepancy surfaces at an arbitrary later time, when the cause can no longer be reconstructed. Three defects of exactly this kind are recounted here from our own work on PostgreSQL: an inconsistent comparator in sorted index construction, the premature return of deleted pages to circulation, and prepared transactions overlooked when building an index concurrently. None of them is detectable by functional testing, because every individual operation returns a correct result; what fails is a property of the structure as a whole.

We report the structural invariants of the generalized search tree and the generalized inverted index, stated in terms of the operator class interface so that they hold for any key type, and a verification procedure for them that is usable in production: on a live system, without exclusive access to the index, with memory bounded by the width of one tree level, and on a physical replica so that the load falls somewhere other than on the system serving users. We describe the two techniques that make this possible — a level-order traversal that carries parent keys downward through intermediate storage, and sibling-link verification with lock coupling in a single direction — and the normalization of index tuples without which comparing a tuple against a freshly built one compares representations rather than values.

We also report that the checker itself needed checking. Four defects have been found in it since release, and three of them were not false alarms but the opposite: guards written so that the check behind them almost never ran, so that the tool reported success on indexes it had not examined. That is the failure mode it exists to detect, reappearing one level up, and it is the harder of the two to notice, because nobody investigates a clean bill of health.

1 Introduction

An index is a redundant structure: everything it contains is derivable from the table it indexes. That redundancy is what makes it fast, and it is also what makes its failures quiet. If the index and the table disagree, the database does not stop; it answers a query with a subset of the correct answer, and no layer of the system is in a position to notice.

This is not a hypothetical failure mode. Three defects from our own work on PostgreSQL’s generalized search tree fall squarely into it. A comparator used in sorted construction was inconsistent for keys that mapped to equal values, producing a tree whose ordering assumptions did not hold. Deleted pages could be returned to circulation before every scan that might reach them had finished, so a scan could read a page holding unrelated data. And building an index concurrently did not account for prepared transactions, so rows committed through two-phase commit could be silently absent from it. In each case every individual operation returned a correct result. What was violated was a property of the structure as a whole, and no test of individual operations can observe such a property.

The conclusion we draw is that a class of correctness properties in a storage engine needs a different discipline from testing behaviour: state the invariants of the structure, and check them against the actual bytes on disk. That conclusion is not novel in itself — it is the premise of amcheck for the B-tree [4]. What this paper contributes is its extension to an extensible index framework, where the key type is supplied by the user and the checker cannot know what a key means, and its extension to production conditions, where the check must run without stopping the workload.

Contributions.

1. Structural invariants for the generalized search tree and the generalized inverted index, expressed entirely through the operator class interface and therefore valid for any key type (Section 2).
2. A level-order traversal that verifies parent key consistency with memory bounded by the width of one tree level rather than by the size of the index (Section 3).
3. Sibling-link verification with lock coupling, and the argument that makes coupling admissible in a checker (Section 4).
4. Normalization of index tuples, without which comparison against a regenerated tuple compares representations rather than values (Section 5).
5. An account of running the check on a physical replica, and of the defects found in the checker itself after release (Sections 6 and 8).

2 Why the operator class interface is the right vocabulary

A generalized search tree [5] is a balanced tree whose node occupies a page and whose entries pair a key with a downlink, the key covering every key in the subtree below. The key type and its operations belong to an operator class supplied outside the framework: `consistent`, `union`, `penalty`, `picksplit` and the rest. The framework does not know whether a key is a rectangle, a signature or a range, and a verification tool built into the framework cannot know either.

This is a constraint, and it is also the design principle. If the invariants are stated using only the operator class interface, the checker inherits the extensibility of the framework: it works for a key type written after the checker was. If they are stated in terms of any particular key semantics, it does not. Every invariant below is therefore expressed through `consistent` alone, which is the operation the search itself uses.

1. *Parent key consistency.* For every entry of an internal page, the key of that entry covers the keys of all entries on the page it points to: $\text{consistent}(k_{\text{parent}}, k_{\text{child}})$ holds for every child. A violation means a search will not descend into a subtree that contains an answer — silently incomplete results, the failure mode of Section 1.
2. *Balance.* All leaf pages lie at the same depth, and an internal page points either only to internal pages or only to leaf pages.
3. *Completeness of the right-link chain.* A page's right link points to a page of the same level, and following the chain visits every page of the level exactly once.
4. *Sibling agreement.* The right link of a left neighbour points to a page whose left link points back to it.
5. *Unreachability of deleted pages.* No entry of any internal page points to a page marked deleted.

Invariant 3 has a consequence for construction that is worth stating explicitly, because it runs the other way from how such things are usually decided. Bulk loading a tree bottom-up could leave right links unset, since no search needs them: a search descends. Leaving them unset would force the checker to distinguish indexes by how they were built, and an invariant that holds only for some instances of a structure is not an invariant. The construction was therefore made to set them. Verifiability is here a design input rather than an afterthought.

For the inverted index the object is not a tree but a graph: an entry tree whose leaves hold either inline posting lists or the roots of posting trees. The invariants restate accordingly — key consistency along paths of the graph, at least one downlink per internal page, uniformity of the level an internal page points to, and ordering of entries within a page.

3 Verifying parent keys without holding the index

The direct way to check invariant 1 is to descend from the root, carrying each parent key to its children. This is correct and unusable in production for two reasons: the descent reads pages in an order unrelated to their placement, and it holds state proportional to the width of the level being descended into, which for a large index means holding most of the index.

The procedure used instead traverses in level order and passes parent keys downward through intermediate storage. Memory is then bounded by the width of one level rather than by the size of the index, and the pages of each level are read in increasing page number — the same argument as for maintenance, and for the same reason.

The check runs under share locks and does not block readers or writers. It therefore observes a moving structure, and its invariants must be ones that hold of every intermediate state a concurrent modification passes through, not merely of quiescent states. This is a real restriction on what can be checked, and it is the reason invariant 1 is stated as coverage rather than as equality: a concurrent insertion may have already widened a parent key without yet having inserted the entry that motivated it, and coverage survives that window where equality would not.

4 Sibling links and lock coupling

Invariant 4 cannot be checked while holding a lock on one page: the check needs to see a left neighbour and its right neighbour at once. That means lock coupling — holding one page’s lock while acquiring the next — which integrity tools had previously avoided, on the reasonable grounds that it complicates reasoning about deadlock.

The argument for admitting it here has two halves. Formally, the locks are taken strictly in one direction, left to right, so no cycle of waits can form with any other party that also respects that order; and they are share locks, so readers are unaffected. Practically, sibling verification detects a substantial share of the corruption actually encountered in the field at very low cost, which is what justifies the additional complexity. We state both halves because the first alone would not have been enough to change the practice, and the second alone would not have been enough to make it safe.

Implemented in 39132b784ae, released in PostgreSQL 13; discussed in [2].

5 Normalizing tuples before comparison

The principal way to check that an index agrees with its table is to take an index tuple and compare it against a tuple built afresh from the table value. Byte comparison does not work. The same value of a variable-length type may be represented in several ways — compressed or not, with headers of different lengths — and which representation is present depends on the history of operations that produced it, not on the value. A checker comparing bytes reports differences that are not differences.

The fix is to normalize both tuples to a canonical representation before comparing: `b1fe8efdf17` for uncompressed variable-length data, and `ab65dfb0fb2` for the differing header sizes of short variable-length data.

The sequel belongs in this paper rather than in a footnote. The normalization itself then turned out to use the wrong accessor for the size of a variable-length datum (`da1eff08a5b`), which is to say that the code written to remove a source of false positives introduced one. We return to this in Section 8.

6 Running the check on a replica

Verification reads the whole index, and a system serving users is the wrong place to do that. The practical requirement is therefore that the check run on a physical replica, which shifts the load off the primary and makes it affordable to run often. Two things had to be corrected for that to work.

Relations that are not fully logged have no meaningful content on a replica, and checking them produces reports that mean nothing; they are skipped (6754fe65a4c). And a check that needs a consistent view of the data must obtain one under recovery conditions, where the usual mechanism does not apply unchanged (1f28982e409).

There is a subtler dependency. On a replica the structure is the product of replaying the write-ahead log, so the correctness of the check is contingent on the correctness of that replay. Without the corresponding fixes, the check reports corruption that exists only in the replica's reconstruction of the index. This is worth stating plainly: verification on a replica checks the primary's index and the replay path at once, and does not by itself tell you which of them is at fault.

7 Making corruption a machine-readable event

A report of corruption is only useful to an operator if a program can act on it. Distinguishing corruption by the text of an error message is not something one can build automation on. Corruption reports were therefore given error codes of their own (8ec97e78a77), so that tooling can recognize the condition and, for instance, take a node out of service without a human reading a log.

This is the smallest change discussed in this paper and, measured by what it enables, not the least useful: it is what turns verification from a thing a person runs during an incident into a thing a system runs continuously.

8 The checker needed checking

Verification of the inverted index shipped in PostgreSQL 18 (14ffaace0fb, with common routines factored out in d70b17636dd). Since then four defects have been fixed in it: posting tree checks (0cf205e122a), the parent key check (cdd1a431f21), the ordering of entry-tree items (0b54b392334), and a memory leak in the parent key consistency pass reported from outside [6]. Support for the generalized search tree is partial and still in review [3], seven years after the first attempt at it [1].

We report this deliberately, because the shape of it matters, and it is not the shape we expected before looking.

Three of the four were not false alarms. They were the opposite: checks that silently did not check. The clearest is the parent key check (cdd1a431f21). It guarded its work with a test on the page's right link, written as a test for the link being zero — but an absent right link is recorded as the largest representable block number, not as zero. The guard was therefore almost always false, the parent key was left unset, and no comparison against it ever ran. The function reported success on indexes it had not examined. The ordering check for the entry tree skipped comparisons across attributes of a multi-column index (0b54b392334); the posting tree check invalidated parent keys for more pages than necessary and so declined to check them (0cf205e122a). In each case the tool answered “no corruption found” without having looked.

That is precisely the failure mode of Section 1, reappearing one level up. The index answers a query with a subset of the correct answer; the checker answers the question “is this index sound” with a subset of the checks it claims to perform. Both are silent, both are invisible to functional tests — a test that runs the checker on a good index and expects no complaint passes either way — and both are found only by someone reading the code or by an independent implementation disagreeing. It was in fact a reader: the posting tree issues were reported from outside by Arseniy Mukhin, as was the memory leak in the parent key pass [6]. The normalization defect (da1eff08a5b), a variable-length size accessor used before the header size was known, was reported by Andres Freund.

We had expected, before assembling this list, that the dangerous defect in an integrity tool would be the false positive: an alarm on a sound index sends the operator hunting for damage that does not exist, and after that happens twice the tool stops being run, at which point its detection rate is zero however good it is. That reasoning still seems right to us, and it is not what the record shows happened. What happened

is that the tool was too quiet, not too loud, and being too quiet is harder to notice by construction: nobody investigates a clean bill of health.

It follows that a verification tool is subject to the argument it exists to make. Its own soundness is a structural property that no functional test of its callers can observe, and the evidence for it is the same kind of evidence: deployment across many key types and workloads, over time, plus readers. Seven years from the first patch [1] to partial support in review is not obviously too slow for something that has to meet that standard.

9 Evaluation

10 Limitations

Verification of the generalized search tree is partial: what is committed covers the inverted index, and the tree is still in review. The check runs concurrently and therefore verifies only invariants that hold of every intermediate state, so properties true merely of quiescent structures are outside its reach. Coverage of key types rests on the operator class interface being used as intended; an operator class whose `consistent` is itself wrong will be found consistent with itself. And on a replica the check cannot separate a defect in the index from a defect in log replay.

11 Conclusion

The failures that matter most in an index access method are the ones that do not look like failures. Every operation succeeds, the structure quietly stops agreeing with the table, and nothing in the ordinary path of the system is in a position to notice. Testing behaviour cannot reach this class, because the behaviour is correct at every step.

What reaches it is stating the invariants of the structure and checking the bytes against them. For an extensible framework this can be done without knowing what a key means, using only the interface the search itself uses, which is what makes the checker apply to key types written after it. Doing it on a running system constrains which invariants are available — only those that hold of every intermediate state — and doing it on a replica makes it affordable enough to run continuously rather than during incidents.

The last finding is the one we would least like to be true. The checker is not exempt from its own argument: its soundness is exactly the kind of property it was built to check, and it failed in exactly the way it was built to detect. Three of the four defects found in its first year were checks that quietly did not run and reported success on indexes they had not examined — silent incompleteness, one level up from the silent incompleteness the tool exists to catch. We had expected the characteristic failure of an integrity tool to be the false alarm, which destroys trust and gets the tool switched off. The record says otherwise, and says something less comfortable: the tool was too quiet, and being too quiet is the harder fault to find, because nobody investigates a clean bill of health.

Acknowledgements

Список литературы

- [1] Andrey Borodin et al. `amcheck` verification for GiST. Discussion on the `pgsql-hackers` mailing list, 2018.
- [2] Andrey Borodin et al. `Amcheck`: do rightlink verification with lock coupling. Discussion on the `pgsql-hackers` mailing list, 2019.
- [3] Andrey Borodin et al. `amcheck`: support for GiST. Discussion on the `pgsql-hackers` mailing list, 2025.

- [4] Peter Geoghegan et al. amcheck (B-tree integrity checking tool). Discussion on the postgresql-hackers mailing list, 2016.
- [5] Joseph M Hellerstein, Jeffrey F Naughton, and Avi Pfeffer. *Generalized search trees for database systems*. University of Wisconsin-Madison, 1995.
- [6] Arseniy Mukhin et al. GIN amcheck leaks memory in gin_check_parent_keys_consistency. Discussion on the postgresql-hackers mailing list, 2026.